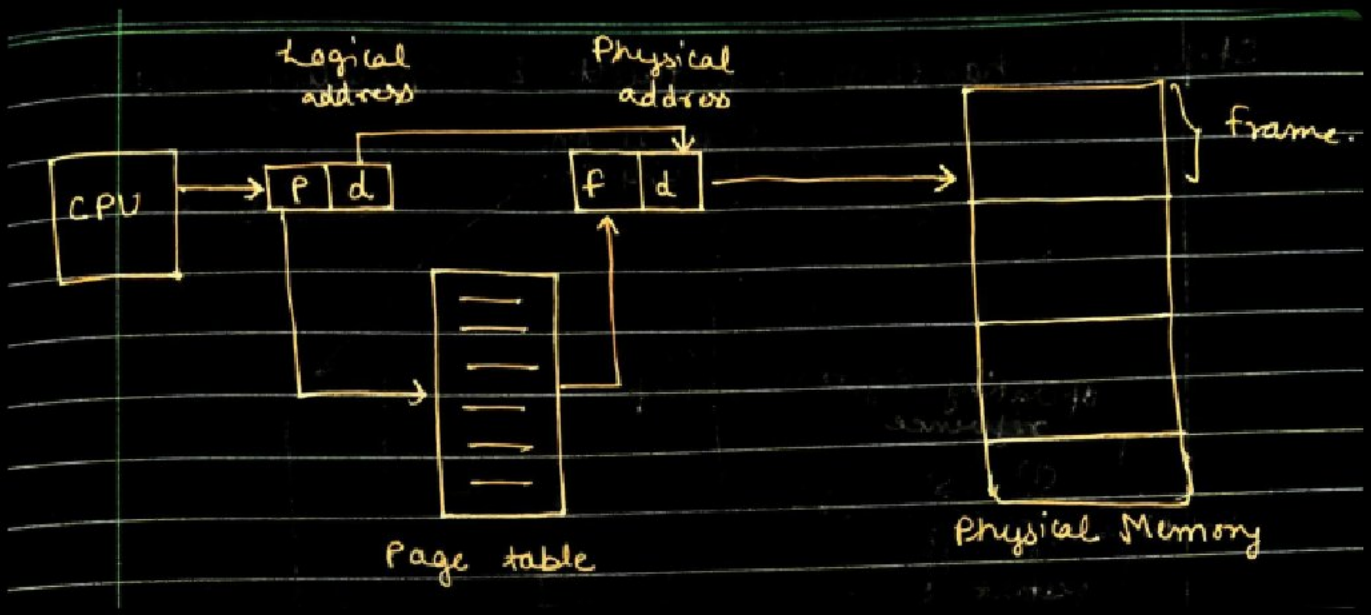
Paging

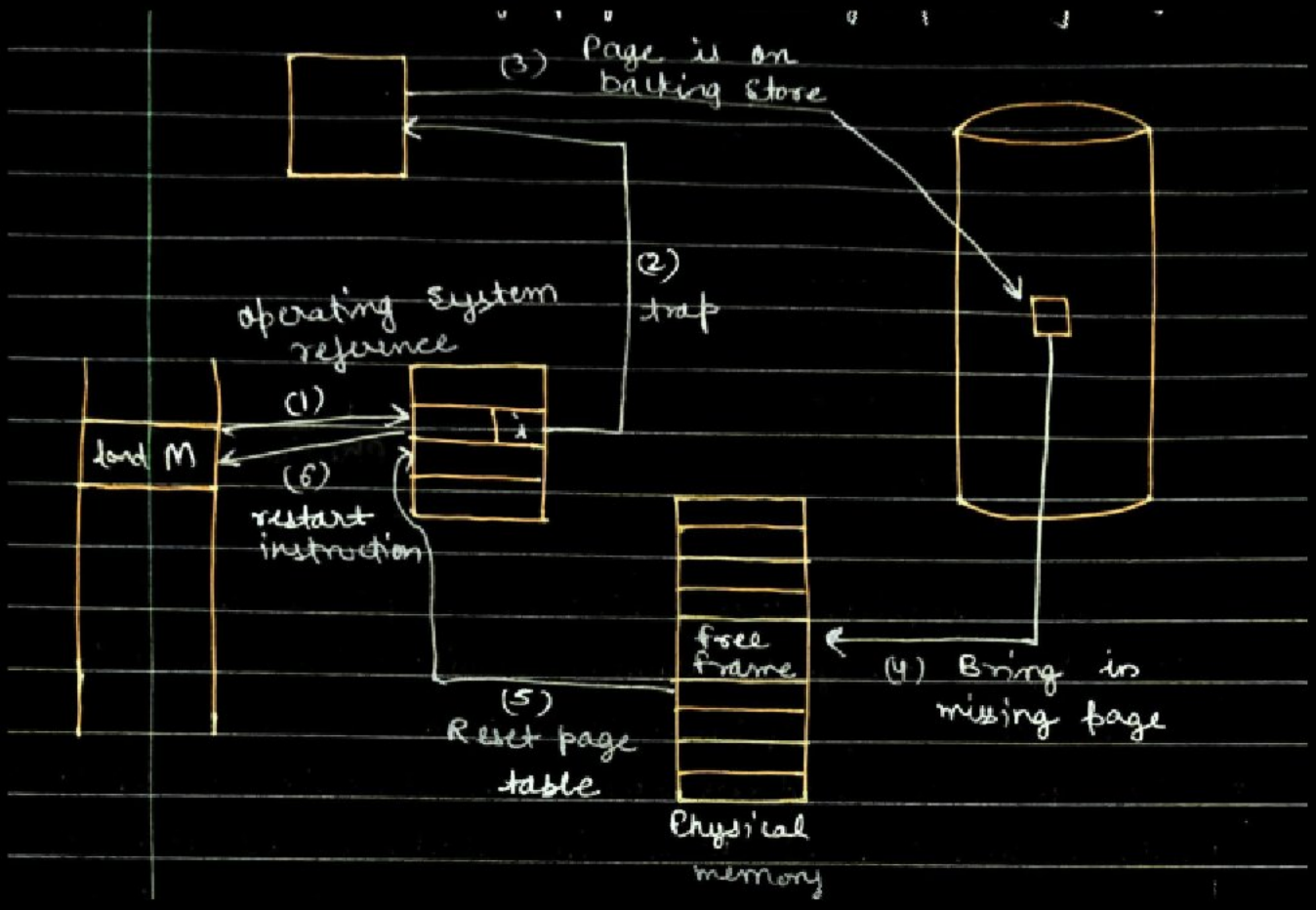
Paging is a memory management scheme that is essential for modern multitasking OSs because it **eliminates the need for contiguous allocation of physical memory**. This allows a process to reside in physical memory non-contiguously, making use of all available free space.

- **Core Idea:** Divide both memory spaces into fixed-size blocks.
  - **Physical Address Space** (RAM) is divided into equal-sized blocks called **frames**.
  - **Logical Address Space** (the program's view) is divided into equal-sized blocks called **pages**.
  - Crucially, **Page Size = Frame Size**.
- **Addresses:**
  - **Logical Address (LA):** An address generated by the **CPU**. The set of all LAs is the **Logical Address Space**.
  - **Physical Address (PA):** An address that is actually available on the **memory unit (RAM)**. The set of all PAs is the **Physical Address Space**.
- **The Mapping:** The translation from the Logical Address to the Physical Address is performed dynamically (at run-time) by the **Memory Management Unit (MMU)**, which is a specialized hardware device. This mapping is known as the **paging technique**.
- **Fragmentation:** Paging **avoids external fragmentation** because any free frame can hold any page. However, it can still cause **internal fragmentation** (since processes rarely fit perfectly into an exact number of pages, the last page is often partially unused).



Page Fault or Page Error

A **Page Fault** is an interrupt that occurs when a program tries to access a page that is logically part of its address space but is **currently not available in main memory (RAM)**.



- **Process:**

1. The CPU generates a Logical Address.
2. The MMU looks up the corresponding entry in the **Page Table**.
3. If the **valid/invalid bit** in the Page Table entry is marked **invalid** (meaning the page is on disk, not in RAM), an interrupt is triggered.

- **Action:** The interrupt triggers the OS to:
  1. Put the interrupted process into a **blocking state**.
  2. Fetch the required page from **virtual memory** (disk) and load it into an available frame in RAM.
  3. Update the **Page Table** to show the page's new physical frame location and mark the entry as valid.
  4. Place the process back into the **ready state** to continue execution.
- **Invalid Page Error:** Occurs if the OS cannot find the page in virtual memory (or if the address is truly illegal, such as accessing memory outside the process's allocated space).

Segmentation

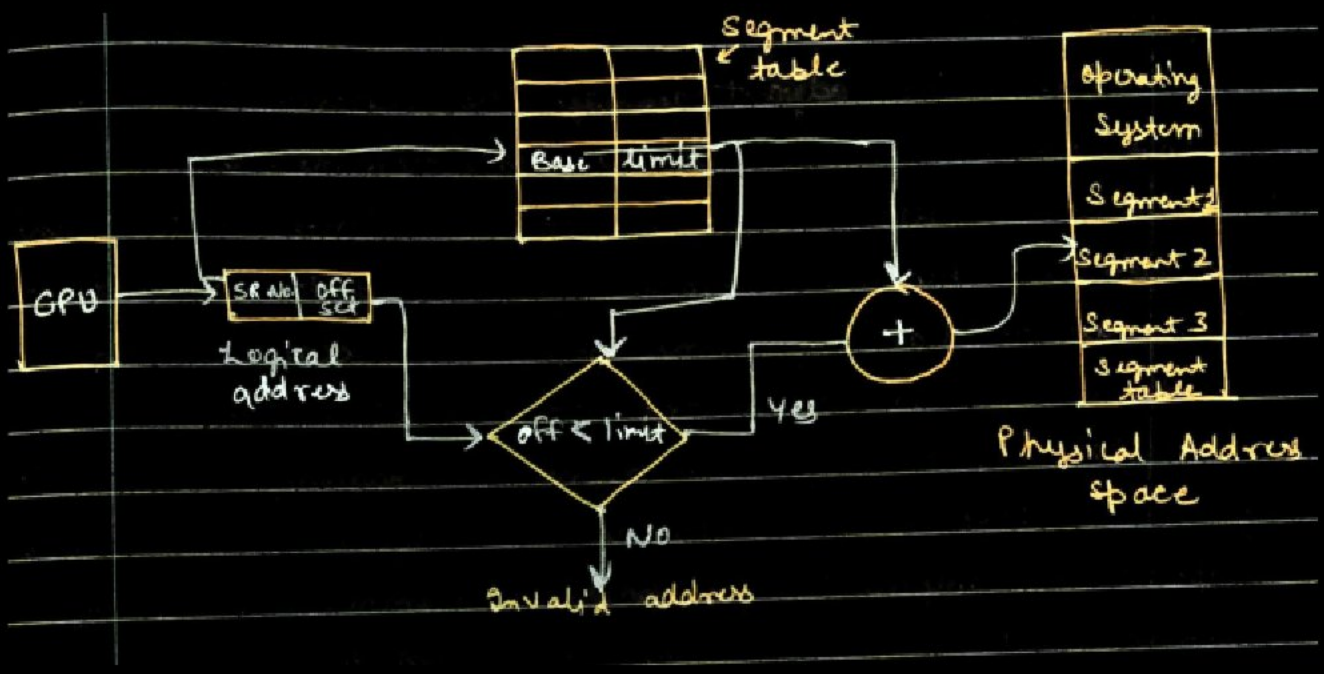
Segmentation is a memory management scheme that supports the **user's view of memory**—a program is viewed as a collection of logical segments rather than fixed-size pages.

- **Logical Unit:** A segment is a **logical unit** of variable size (e.g., the main program, a function, a stack, an array, a symbol table).
- **Address Structure:** Each address generated by the CPU is divided into two parts: a **segment number** and an **offset**.
- **Mapping:** The **segment number** is used as an index into the **Segment Table**.
  - Each entry in the Segment Table contains two vital pieces of information:
    - **Base Address:** The starting physical address where the segment resides in memory.
    - **Limit:** Specifies the length of the segment.
- **Address Translation:** The offset (the logical address within the segment) must be checked against the **limit**. If the offset is less than the limit, the physical address is

calculated as:

$$\text{Physical Address} = \text{Base Address} + \text{Offset}$$

If the offset is greater than the limit, a memory access violation occurs.



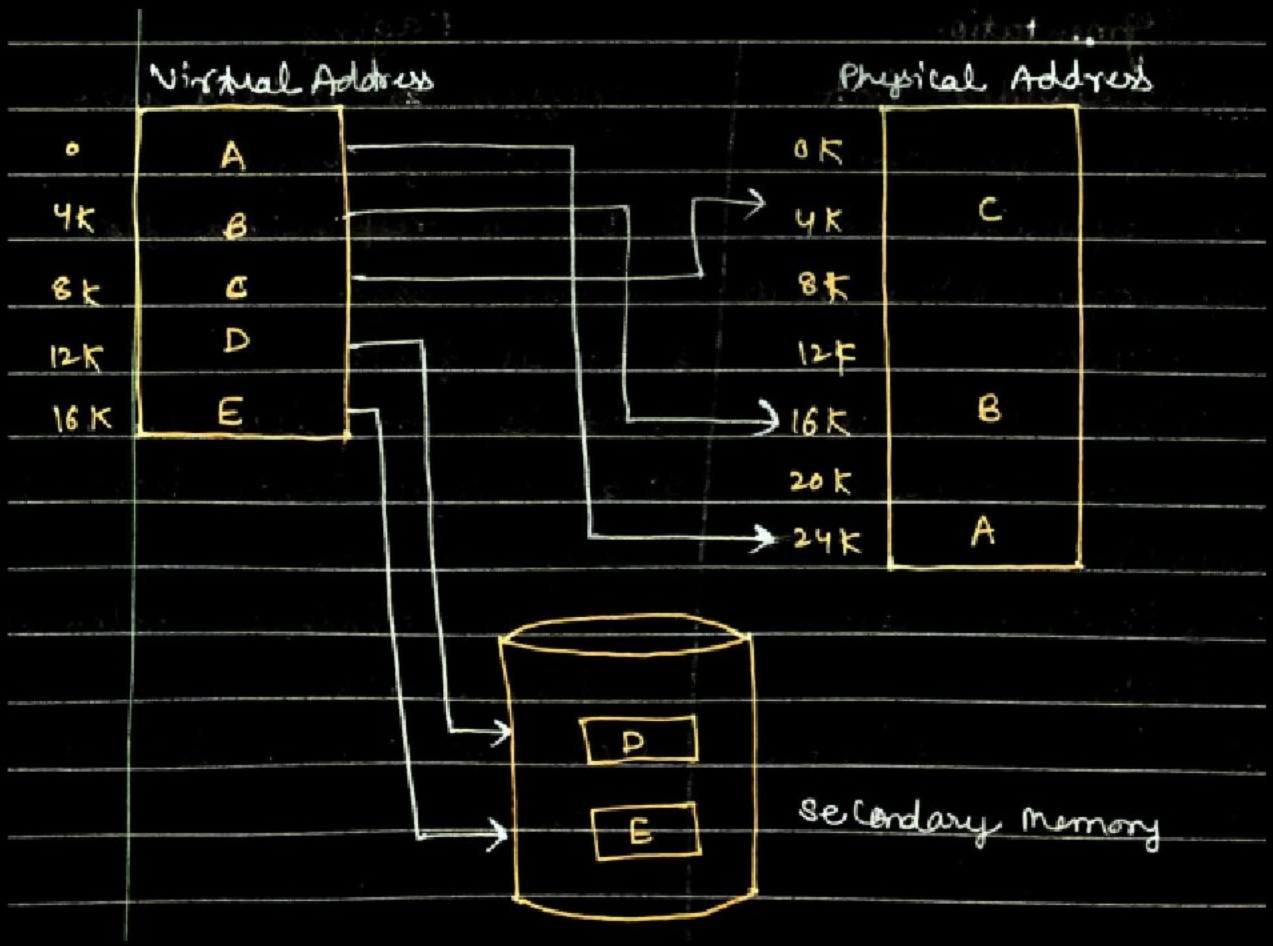
Paging vs. Segmentation

Feature	Paging	Segmentation
User View	Invisible to the user (done by OS)	Visible to the user (based on program structure)
Division	Fixed-size pages	Variable-size segments
Internal Fragmentation	Yes (in the last page)	No
External Fragmentation	No (solved by fixed frame size)	Yes (due to variable-sized segments)
Address	Page Number, Offset	Segment Number, Offset

Virtual Memory

Virtual Memory (VM) is a memory management technique that allows a computer to address **more memory than the amount physically installed** (RAM).

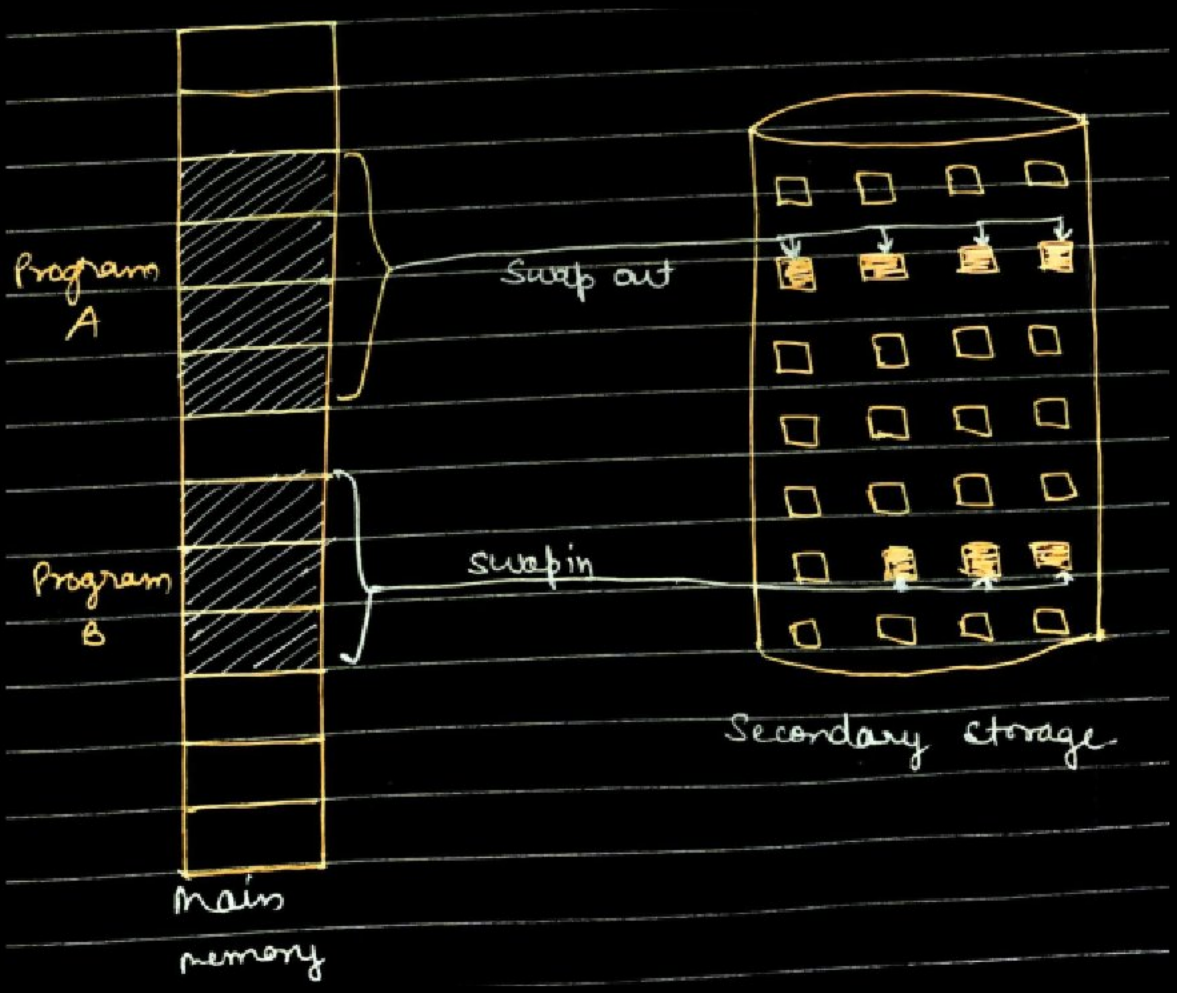
- **Core Concept:** It creates the illusion of a huge, uniform memory space for each process. This "extra" memory is actually a section of the **hard disk** (known as the swap space or backing store) that is set up to simulate the computer's RAM.
- **Advantages:**
  1. **Enables Large Programs:** Programs can be larger than the physical memory available.
  2. **Increased Multiprogramming:** Only parts of a process need to be in RAM, freeing space for more processes.
  3. **Memory Protection:** Translation from the virtual address to the physical address provides inherent memory protection.
- **Mechanism:** VM is implemented using both **hardware** (MMU for translation) and **software** (OS for managing the swap space). The primary method used to implement VM is **Demand Paging**.



Demand Paging

Demand Paging is the process of loading the pages of a program into memory **only when they are actually referenced** (on demand), typically when a **page fault** occurs.

- **Benefit:** This avoids loading pages that are never used, significantly speeding up process startup and reducing RAM usage.
- **Process Recap:** If the CPU tries to reference a page not in RAM, a **page fault** occurs. The OS then searches the disk, brings the required page into a free frame, updates the page table, and restarts the process.



Page Replacement Algorithms

When a page fault occurs, and there are **no free frames** in RAM, the OS must select an existing page in memory to swap out to the disk to make room for the new demanded page. This selection is governed by a **Page Replacement Algorithm**.

1. First In First Out (FIFO)

- **Principle:** The algorithm selects the page that has been in memory for the **longest time** (the oldest page) for replacement.
- **Mechanism:** It uses a FIFO queue. The page at the **head of the queue** is the replacement candidate. When a new page comes in, it's inserted at the tail.
- **Drawback:** It can perform poorly because the oldest page might actually be a heavily used page (e.g., system libraries), leading to frequent, unnecessary page faults (known as **Belady's Anomaly**).

2. Optimal Replacement Algorithm (OPT or MIN)

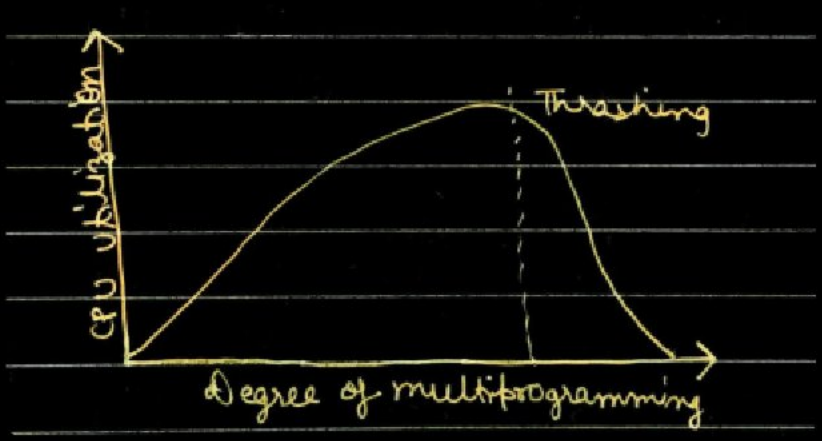
- **Principle:** Selects the page for replacement that will **not be used for the longest period of time** in the future.
- **Performance:** It guarantees the **lowest possible page fault rate** of all algorithms.
- **Implementation: Impossible to implement** in practice because it requires the OS to have **perfect knowledge of future events** (i.e., the process's entire future sequence of memory accesses). It is used as a benchmark for comparison.

3. Least Recently Used (LRU)

- **Principle:** Selects the page that has **not been used for the longest period of time** in the recent past.
- **Justification:** It uses the **recent past as an approximation of the near future** (if a page hasn't been used recently, it's less likely to be used soon).
- **Mechanism:** Requires complex hardware support (like counters or time-stamps) to record the time of the last use for every page. When a fault occurs, the OS checks all time-stamps to find the oldest one.

Thrashing

Thrashing is a pathological state that occurs when a system spends **more time servicing page faults and swapping pages** (doing I/O) than executing actual instructions.



- **Cause:** It usually happens when the **degree of multiprogramming (DOP)** is too high, and the total memory required by the currently active processes exceeds the available physical RAM.
- **Vicious Cycle:**
  1. The system starts running many processes (High DOP).
  2. Processes immediately need pages, causing frequent **page faults**.
  3. The OS swaps out pages to bring new ones in.
  4. Since there isn't enough memory for all running processes' minimum needs, the swapped-out page is often needed immediately by another process (or the same process), leading to yet another fault.
  5. The system is caught in an endless loop of swapping pages in and out (**paging**).
- **Effect:**
  - **CPU becomes idle** or severely underutilized because it is waiting for the slow disk I/O to complete the page transfers.
  - The OS sees the CPU is idle, and mistakenly tries to increase utilization by increasing the **degree of multiprogramming** (loading even *more* processes), which only makes the thrashing worse.
- **Solution:** The OS must recognize thrashing and **reduce the degree of multiprogramming** (suspend or swap out some processes entirely) to provide more

frames to the remaining active processes.